GLO-3100 – Cryptographie et sécurité informatique

Exercice 3 : Les modes d'opération, en vrai

Guide de démarrage

Ce guide vous donne les notions et les outils nécessaires pour démarrer l'exercice 3 : ce qu'il faut savoir sur AES, à quoi servent les *modes d'opération* et le *remplissage*, comment utiliser une bibliothèque cryptographique et comment aborder le défi de la question 2.

Le langage est à votre choix

Comme mentionné dans l'énoncé du TP1, vous pouvez réaliser l'exercice en **Python**, en **C/C++** ou en **Java**, avec la bibliothèque standard (ou une bibliothèque reconnue) de votre langage (`cryptography` en Python, `Crypto++` en C++, `javax.crypto` en Java, OpenSSL en C). Les exemples de ce guide sont écrits en Python parce qu'ils sont courts et faciles à lire, mais les notions sont **exactement les mêmes** dans les autres langages. La section 4.7 donne les équivalents en Java, C++ et C.

Table des matières

1 Prérequis **2**
1.1 Ce qu'il faut installer . . . 2
1.2 Manipuler des octets (exemples en Python) . . . 2

2 AES : ce qu'il faut savoir **3**
2.1 Un chiffrement symétrique . . . 3
2.2 Un chiffrement par blocs . . . 3
2.3 La taille de la clé est variable . . . 3

3 Le problème : un message fait rarement 16 octets **4**
3.1 Les modes d'opération . . . 4
3.2 Le remplissage (*padding*) PKCS#7 . . . 5
3.3 L'en-tête du fichier BMP . . . 6

4 Utiliser une bibliothèque : l'exemple de `cryptography` (Python) **6**
4.1 Les imports . . . 6
4.2 Construire un chiffreur . . . 6
4.3 Chiffrer et déchiffrer . . . 6
4.4 L'erreur classique : oublier le remplissage . . . 7
4.5 Ajouter et retirer le remplissage PKCS#7 . . . 7
4.6 Une petite expérience à faire . . . 7
4.7 Équivalences en Java, C++ et C . . . 8

5 Démarrer la question 1 : l'expérience du pingouin **9**

6 Démarrer la question 2 : le défi **9**
6.1 Le XOR (« ou exclusif », noté ⊕) 9
6.2 La situation 10
6.3 Lire le fichier `defi.txt` 10
6.4 Le principe du *crib dragging*, sur un exemple 10
6.5 Une démarche possible 11

7 Ce qu'il faut rendre **11**

1 Prérequis

1.1 Ce qu'il faut installer

Selon le langage choisi :

— **Python** : Python 3 (version 3.8 ou plus récente) et la bibliothèque `cryptography` :

```
pip install cryptography
python -c "import cryptography; print(cryptography.__version__)" # vérification
```

— **Java** : un JDK récent (11 ou plus). Le paquet `javax.crypto` est inclus dans le JDK : rien d'autre à installer.

— **C++** : la bibliothèque Crypto++ (paquet `libcrypto++-dev` sous Debian/Ubuntu, `cryptopp` avec Homebrew sous macOS, ou `vcpkg` sous Windows).

— **C** : la bibliothèque OpenSSL (`libcrypto`), via l'interface `EVP`.

Il vous faut aussi une visionneuse d'images capable d'ouvrir un fichier `.bmp` (celle de votre système d'exploitation suffit). Placez vos programmes dans le même dossier que `image_secrete.bmp` et `defi.txt`.

1.2 Manipuler des octets (exemples en Python)

Toute la cryptographie travaille sur des **octets** et pas sur des chaînes de caractères. Un octet est un nombre entre 0 et 255. En Python, une suite d'octets a le type `bytes` ; en Java, c'est un tableau `byte[]` ; en C/C++, un tableau d'`unsigned char` (ou un `std::string`/`SecByteBlock` avec Crypto++).

```
donnees = b"Bonjour" # le préfixe b indique des octets
print(len(donnees)) # 7 : nombre d'octets
print(donnees[0]) # 66 : un octet est un entier (code de 'B')
print(donnees.hex()) # '426f6e6a6f7572' : affichage en hexadécimal
print(bytes.fromhex("426f6e")) # b'Bon' : l'opération inverse

texte = "Bonjour" # une chaîne (type str)
octets = texte.encode() # str -> bytes
retour = octets.decode() # bytes -> str
```

Découper et coller (*slicing*). On extrait une partie d'une suite d'octets avec des crochets et on colle deux suites avec `+` :

```
donnees = b"ABCDEFGHIJ"
print(donnees[:3]) # b'ABC' : du début jusqu'à l'indice 3 EXCLU
print(donnees[3:]) # b'DEFGHIJ' : de l'indice 3 jusqu'à la fin
print(donnees[2:5]) # b'CDE'
print(b"AB" + b"CD") # b'ABCD' : concaténation
```

Lire et écrire un fichier binaire. Une image n'est pas du texte : on ouvre le fichier en mode **binaire** (en Python, `"rb"` pour lire, `"wb"` pour écrire ; en C/C++, le mode `"rb"` de `fopen` ou `std::ios::binary` ; en Java, `Files.readAllBytes` et `Files.write`).

```
with open("un_fichier.bin", "rb") as f:
    contenu = f.read() # contenu est de type bytes

with open("copie.bin", "wb") as f:
    f.write(contenu)
```

Générer des octets aléatoires. Pour une clé ou un IV, il faut un aléa de qualité *cryptographique*. En Python, on utilise le module `secrets`, conçu pour cet usage (`os.urandom` est équivalent) et **jamais** le module `random` (prévisible). Les équivalents sont `SecureRandom` en Java (pas `java.util.Random`), `AutoSeededRandomPool` avec Crypto++ et `RAND_bytes` avec OpenSSL (pas `rand()`).

```
import secrets

a = secrets.token_bytes(16) # 16 octets aléatoires
```

2 AES : ce qu'il faut savoir

2.1 Un chiffrement symétrique

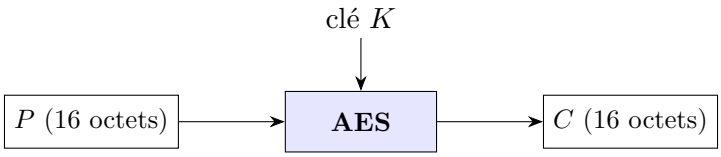
AES (*Advanced Encryption Standard*) est un algorithme de chiffrement **symétrique** : la *même* clé sert à chiffrer et à déchiffrer. Celui qui chiffre et celui qui déchiffre doivent donc partager cette clé secrète.

Ne pas confondre

AES n'est **pas** un chiffrement asymétrique. En cryptographie asymétrique (RSA, par exemple que nous verrons plus tard), il y a deux clés différentes : une clé publique pour chiffrer et une clé privée pour déchiffrer. Avec AES, il n'y a qu'une seule clé, qui doit rester secrète.

2.2 Un chiffrement par blocs

AES est un chiffrement **par blocs**. C'est une fonction qui prend en entrée :
— un **bloc** de **128 bits**, soit **16 octets**, *exactement* ;
— une **clé** K,
et qui renvoie un bloc chiffré de 128 bits (16 octets). On note $C = E_K(P)$, où P est le bloc en clair et C le bloc chiffré. Le déchiffrement fait l'inverse : $P = D_K(C)$.



2.3 La taille de la clé est variable

La taille du *bloc* est toujours de 128 bits, mais la taille de la *clé* peut varier :

Variante	Taille de clé	Taille du bloc	Nombre de tours internes
AES-128	128 bits (16 octets)	128 bits	10
AES-192	192 bits (24 octets)	128 bits	12
AES-256	256 bits (32 octets)	128 bits	14

Le nombre dans le nom (AES-**128**) désigne donc la taille de la **clé**, pas celle du bloc. Dans cet exercice, on utilise **AES-128** : il vous faut une clé de **16 octets**.

Règle d'or

On n'écrit jamais soi-même AES : on utilise l'implémentation d'une bibliothèque reconnue. Votre travail consiste à *bien utiliser* cette bibliothèque, pas à réimplémenter AES.

3 Le problème : un message fait rarement 16 octets

AES ne sait chiffrer qu'**un seul bloc de 16 octets**. Or l'image de l'exercice fait plusieurs centaines de milliers d'octets. Deux questions se posent :

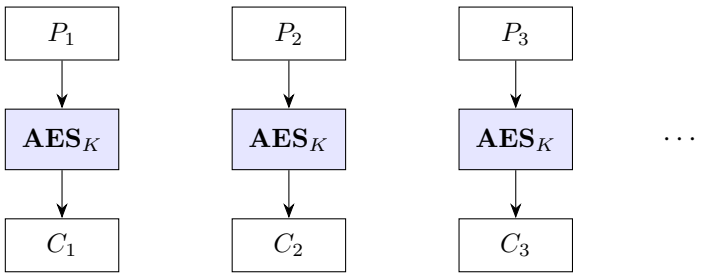
1. **Comment enchaîner** les nombreux blocs d'un long message ? C'est le rôle du **mode d'opération**.
2. **Que faire si la longueur n'est pas un multiple de 16** (le dernier bloc est incomplet) ? C'est le rôle du **remplissage** (*padding*).

3.1 Les modes d'opération

Un mode d'opération est une *règle* qui explique comment utiliser AES (qui ne traite qu'un bloc) pour chiffrer un message de longueur quelconque. On découpe d'abord le message en blocs de 16 octets : $P_1, P_2, \ldots, P_n$. Le mode dit ensuite comment calculer les blocs chiffrés $C_1, C_2, \ldots, C_n$. Il existe plusieurs modes (ECB, CBC, CTR, GCM…) ; l'exercice vous demande de comparer les deux premiers.

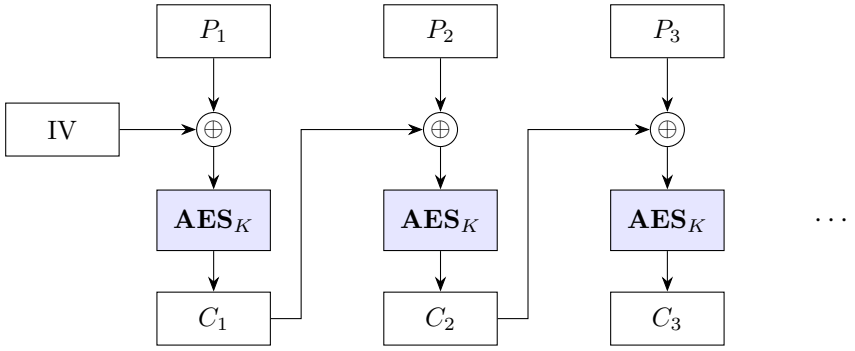
Le mode ECB (*Electronic Codebook*). On chiffre chaque bloc séparément, avec la même clé :

$$\boxed{C_i = E_K(P_i)}$$



Le mode CBC (*Cipher Block Chaining*). Avant de chiffrer un bloc, on le combine par XOR ($\oplus$, voir section 6.1) avec le bloc chiffré *précédent*. Le premier bloc n'a pas de prédécesseur : on utilise à la place un **vecteur d'initialisation** (IV) de 16 octets aléatoires.

$$\boxed{C_0 = IV, \qquad C_i = E_K(P_i \oplus C_{i-1})}$$



L'IV n'est pas secret (on le transmet en général avec le message chiffré), mais il doit être imprévisible et différent à chaque chiffrement. Sans l'IV, on ne peut pas déchiffrer le premier bloc : conservez-le !

Questions à vous poser (et à tester !)

— Si le message contient deux blocs identiques $P_i = P_j$, que pouvez-vous dire de C_i et C_j en mode ECB ? Et en mode CBC ?

— Une image contient-elle beaucoup de blocs de 16 octets identiques ? Pensez aux grandes zones d'une seule couleur.

Ces deux questions sont au cœur de l'explication demandée à la question 1. La petite expérience de la section 4.6 vous aidera à y répondre.

3.2 Le remplissage (*padding*) PKCS#7

Les modes ECB et CBC exigent que les données aient une longueur **multiple de 16 octets**. Si ce n'est pas le cas, le chiffrement est impossible. Le remplissage consiste à ajouter quelques octets à la fin pour compléter le dernier bloc, d'une façon qui permette de les *retirer sans ambiguïté* après le déchiffrement.

La règle **PKCS#7** : s'il manque n octets pour atteindre un multiple de 16, on ajoute n octets valant chacun n.

Situation	Octets ajoutés (en hexadécimal)
Il manque 3 octets	`03 03 03`
Il manque 1 octet	`01`
Il manque 12 octets	`0c 0c 0c 0c 0c 0c 0c 0c 0c 0c 0c 0c`
Il ne manque rien	un bloc entier : 16 fois `10` (16 s'écrit `10` en hexadécimal)

Exemple sur un message de 13 octets :

$$\underbrace{\texttt{48 65 6c 6c 6f 20 6c 65 20 6d 6f 6e 64}}_{\text{13 octets : « Hello le mond »}}\ \underbrace{\texttt{03 03 03}}_{\text{remplissage}}$$

Pour retirer le remplissage, on lit le *dernier* octet (ici `03`) et on enlève ce nombre d'octets. C'est pour cela qu'on ajoute un bloc complet quand il ne manque rien : sinon, on ne saurait pas si le dernier octet fait partie du message ou du remplissage.

À vous de réfléchir

Calculez le nombre d'octets de pixels de `image_secrete.bmp` (taille du fichier moins l'en-tête). Est-ce un multiple de 16 ? Combien d'octets PKCS#7 va-t-il ajouter ? Quelle sera alors la taille de vos fichiers chiffrés ?

3.3 L'en-tête du fichier BMP

Un fichier BMP commence par un **en-tête** de 54 octets qui décrit l'image (largeur, hauteur, nombre de bits par pixel…), suivi des **pixels**. L'énoncé vous demande de garder cet en-tête **en clair** et de ne chiffrer que la suite :

$$\underbrace{\texttt{en-tête (54 octets)}}_{\text{à garder tel quel}} \;\Big|\; \underbrace{\texttt{pixels}}_{\text{à chiffrer}}$$

Ainsi, la visionneuse sait toujours lire le fichier et elle affiche les pixels chiffrés comme s'il s'agissait d'une image normale.

4 Utiliser une bibliothèque : l'exemple de `cryptography` (Python)

Cette section montre l'utilisation de la bibliothèque Python `cryptography`. Si vous travaillez en Java, en C++ ou en C, lisez-la quand même : les étapes sont les mêmes (choisir l'algorithme, le mode, la clé, l'IV, puis chiffrer) et la section 4.7 vous donne les noms correspondants dans votre langage.

Les exemples ci-dessous portent sur de **petits messages**, pas sur l'image. À vous de les adapter à l'exercice.

4.1 Les imports

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding
```

Le mot `hazmat` signifie « *hazardous materials* » (matières dangereuses) : ce sont les briques de bas niveau, qu'il faut assembler avec soin. C'est exactement ce que demande l'exercice. Dans un vrai projet, on préférerait une interface de plus haut niveau (comme `Fernet` ou `AESGCM`), qui fait les bons choix à votre place.

4.2 Construire un chiffreur

On construit un objet `Cipher` à partir de deux choses : l'**algorithme** (AES avec sa clé) et le **mode**.

```
import secrets

cle = secrets.token_bytes(16) # 16 octets -> AES-128

algo = algorithms.AES(cle) # l'algorithme et sa clé
cipher_ecb = Cipher(algo, modes.ECB()) # mode ECB : pas d'IV

iv = secrets.token_bytes(16) # IV de 16 octets (taille d'un bloc)
cipher_cbc = Cipher(algo, modes.CBC(iv)) # mode CBC : l'IV est obligatoire
```

La taille de la clé passée à `algorithms.AES` choisit la variante : 16 octets pour AES-128, 24 pour AES-192, 32 pour AES-256. Toute autre taille provoque une erreur.

4.3 Chiffrer et déchiffrer

Un `Cipher` fournit un `encryptor()` (pour chiffrer) et un `decryptor()` (pour déchiffrer). On leur donne les données avec `update`, puis on termine avec `finalize` :

```
message = b"exactement 32 octets, pas un de+" # 32 octets = 2 blocs
print(len(message)) # 32
```

```
# Chiffrement
chiffreur = cipher_ecb.encryptor()
chiffre = chiffreur.update(message) + chiffreur.finalize()
print(chiffre.hex())

# Déchiffrement
dechiffreur = cipher_ecb.decryptor()
clair = dechiffreur.update(chiffre) + dechiffreur.finalize()
print(clair) # b'exactement 32 octets, pas un de+'
```

Un objet par opération

Un `encryptor` (ou `decryptor`) ne sert qu'une fois : après `finalize()`, il est inutilisable. Pour un nouveau chiffrement, appelez à nouveau `cipher.encryptor()`.

4.4 L'erreur classique : oublier le remplissage

Essayez de chiffrer un message de 13 octets en ECB :

```
chiffreur = cipher_ecb.encryptor()
chiffreur.update(b"Hello le mond") + chiffreur.finalize()
```

Vous obtenez :

```
ValueError: The length of the provided data is not a multiple of the block length.
```

La bibliothèque `cryptography` n'ajoute **pas** le remplissage toute seule : c'est à vous de le faire. Ce n'est pas le cas partout : en Java, Crypto++ et OpenSSL, le remplissage est ajouté *automatiquement* si vous le demandez (voir section 4.7).

4.5 Ajouter et retirer le remplissage PKCS#7

```
from cryptography.hazmat.primitives import padding

# Attention : la taille de bloc se donne en BITS (128), pas en octets (16)
remplisseur = padding.PKCS7(128).padder()
rempli = remplisseur.update(b"Hello le mond") + remplisseur.finalize()
print(rempli) # b'Hello le mond\x03\x03\x03'
print(len(rempli)) # 16

retireur = padding.PKCS7(128).unpadder()
original = retireur.update(rempli) + retireur.finalize()
print(original) # b'Hello le mond'
```

L'ordre des opérations est donc :

chiffrer : remplir → chiffrer **déchiffrer :** déchiffrer → retirer le remplissage

4.6 Une petite expérience à faire

Avant de vous attaquer à l'image, lancez le programme suivant et **observez** la sortie. Le message est formé de 4 blocs : les blocs 1 et 3 sont identiques, les blocs 2 et 4 aussi.

```
import secrets
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

def afficher_par_blocs(nom, donnees):
    """Affiche les données en hexadécimal, un bloc de 16 octets par ligne."""
```

```
    print(nom)
    for i in range(0, len(donnees), 16):
        print(" bloc", i // 16 + 1, ":", donnees[i:i + 16].hex())

message = b"AAAAAAAAAAAAAAAA" + b"BBBBBBBBBBBBBBBB" \
        + b"AAAAAAAAAAAAAAAA" + b"BBBBBBBBBBBBBBBB" # 64 octets

cle = secrets.token_bytes(16)
iv = secrets.token_bytes(16)

for nom, mode in [("ECB", modes.ECB()), ("CBC", modes.CBC(iv))]:
    chiffreur = Cipher(algorithms.AES(cle), mode).encryptor()
    chiffre = chiffreur.update(message) + chiffreur.finalize()
    afficher_par_blocs(nom, chiffre)
```

À vous de réfléchir

— Comparez les blocs chiffrés 1 et 3, puis 2 et 4, en ECB. Que remarquez-vous ?
— Même question en CBC.
— Relancez le programme : la clé change, les valeurs aussi. Mais ce que vous avez remarqué reste-t-il vrai ?
— Faites le lien avec la formule de chaque mode, puis avec une image qui contient de grandes zones unies.

Si vous travaillez dans un autre langage, refaites cette expérience dans votre langage : c'est un bon premier programme pour prendre en main votre bibliothèque.

4.7 Équivalences en Java, C++ et C

Le tableau suivant donne les noms correspondants dans les bibliothèques des autres langages. Consultez leur documentation pour les détails.

Notion	Java (`javax.crypto`)	C++ (Crypto++)	C (OpenSSL, `EVP`)
Clé AES-128	`new SecretKeySpec(cle, "AES")` (`cle : byte[16]`)	`SecByteBlock cle(16)`	`unsigned char cle[16]`
Aléa crypto.	`new SecureRandom().nextBytes(tab)`	`AutoSeededRandomPool.GenerateBlock(...)`	`RAND_bytes(tab, 16)`
Mode ECB	`Cipher.getInstance("AES/ECB/PKCS5Padding")`	`ECB_Mode<AES>::Encryption`	`EVP_aes_128_ecb()`
Mode CBC + IV	`Cipher.getInstance("AES/CBC/PKCS5Padding")` `new IvParameterSpec(iv)`	`CBC_Mode<AES>::Encryption.SetKeyWithIV(...)`	`EVP_aes_128_cbc()` (IV passé à `EVP_EncryptInit_ex`)
Chiffrer	`init(Cipher.ENCRYPT_MODE, ...)` `doFinal(donnees)`	`StringSource` + `StreamTransformationFilter`	`EVP_EncryptInit_ex` `EVP_EncryptUpdate` `EVP_EncryptFinal_ex`
Remplissage PKCS#7	automatique (`PKCS5Padding`)	automatique (`PKCS_PADDING`, le défaut pour ECB et CBC)	automatique (activé par défaut)

Java : « PKCS5Padding »

En Java, le remplissage s'appelle `PKCS5Padding` pour des raisons historiques. Pour AES (blocs de 16 octets), il s'agit exactement du remplissage PKCS#7 demandé par l'énoncé.

5 Démarrer la question 1 : l'expérience du pingouin

Voici une démarche possible. Chaque étape utilise un outil présenté plus haut.

1. Lire tout le fichier `image_secrete.bmp` en mode binaire.
2. Séparer les 54 premiers octets (l'en-tête) du reste (les pixels).
3. Générer une clé AES-128 aléatoire (16 octets).
4. Appliquer le remplissage PKCS#7 aux pixels.
5. Chiffrer les pixels remplis en mode ECB.
6. Générer un IV aléatoire de 16 octets et chiffrer les pixels remplis en mode CBC.
7. Écrire `image_ecb.bmp` et `image_cbc.bmp` : chacun contient l'en-tête original *suivi* des données chiffrées correspondantes.
8. Ouvrir les deux fichiers dans une visionneuse et comparer avec l'original.

Conseils

— Utilisez la **même clé** pour ECB et CBC : ainsi, la seule différence entre les deux images vient du mode d'opération.
— Vérifiez votre travail : déchiffrez vos données, retirez le remplissage et vérifiez (avec un `assert`, par exemple) que vous retrouvez exactement les pixels de départ.
— Écrivez de petites fonctions (`chiffrer`, `dechiffrer`…) plutôt que de répéter le même code pour chaque mode.

Pièges fréquents

— Ouvrir l'image en mode texte au lieu du mode binaire.
— Chiffrer aussi l'en-tête : la visionneuse ne peut plus ouvrir le fichier.
— Utiliser une clé de mauvaise taille (une chaîne comme `"secret"`, par exemple) : AES-128 attend exactement 16 *octets*.
— Générer la clé ou l'IV avec un générateur non cryptographique (`random` en Python, `java.util.Random` en Java, `rand()` en C/C++).
— *En Python* : donner une taille de bloc de `16` à `padding.PKCS7`, alors qu'elle s'exprime en bits (`128`).
— *En Java, C++ ou C* : ajouter soi-même un remplissage PKCS#7 alors que la bibliothèque en ajoute déjà un (on obtiendrait un remplissage en double).

Pour l'explication demandée, appuyez-vous sur la **définition** du mode ECB (sa formule) et sur ce que vous avez observé dans la petite expérience de la section 4.6.

6 Démarrer la question 2 : le défi

6.1 Le XOR (« ou exclusif », noté $\oplus$)

Le XOR compare deux bits : il vaut 1 s'ils sont différents, 0 s'ils sont identiques.

$$0 \oplus 0 = 0 \qquad 0 \oplus 1 = 1 \qquad 1 \oplus 0 = 1 \qquad 1 \oplus 1 = 0$$

Sur des octets, on applique cette règle bit par bit. En Python, comme en C, C++ et Java, l'opérateur est `^`. Pour deux suites d'octets, on fait le XOR octet par octet, par exemple avec une boucle (en Java : `(byte) (a[i] ^ b[i])`). En Python :

```python
print(0b1100 ^ 0b1010) # 6, soit 0b0110
print(0x41 ^ 0x20) # 97 : 'A' XOR espace donne 'a' !
```

```python
def xor(a, b):
    """XOR octet par octet (s'arrête àla fin de la plus courte des deux suites)."""
    return bytes(x ^ y for x, y in zip(a, b))
```

Propriétés utiles déjà vues en classe (vérifiez-les sur la table ci-dessus) :

$$x \oplus x = 0, \qquad x \oplus 0 = x, \qquad x \oplus y = y \oplus x, \qquad (x \oplus y) \oplus z = x \oplus (y \oplus z).$$

6.2 La situation

Le chiffrement XOR utilise un *flux de clé* k pour chiffrer un message m en calculant $c = m \oplus k$. Dans `defi.txt`, deux messages ont été chiffrés avec **le même** flux de clé :

$$c_1 = m_1 \oplus k, \qquad c_2 = m_2 \oplus k.$$

À vous de réfléchir

— À l'aide des propriétés du XOR, démontrez que $c_1 \oplus c_2 = m_1 \oplus m_2$.
— Pourquoi est-ce une mauvaise nouvelle pour la sécurité ? Que contient $c_1 \oplus c_2$ **et** que ne contient-il plus ?
— Si vous connaissez $m_1 \oplus m_2$ et un morceau de m_1, comment obtenir le morceau de m_2 situé à la même position ?

6.3 Lire le fichier `defi.txt`

Chaque ligne contient un texte chiffré écrit en hexadécimal. Il faut d'abord transformer ces lignes en octets. En Java (17 et plus), `HexFormat.of().parseHex(ligne)` le fait directement ; avec Crypto++, on utilise un `HexDecoder` ; en C, on peut lire deux caractères à la fois avec `sscanf(..., "%2hhx", ...)`. En Python :

```python
with open("defi.txt") as f:
    lignes = f.read().split() # une liste de deux chaînes hexadécimales

c1 = bytes.fromhex(lignes[0])
c2 = bytes.fromhex(lignes[1])
```

6.4 Le principe du *crib dragging*, sur un exemple

Un *crib* est un fragment de texte que l'on *devine* dans l'un des messages (« `␣le` », « `␣que` », « `␣la` »…). On ne sait pas *où* il se trouve, donc on le fait **glisser** sur toutes les positions possibles.

Prenons un exemple **inventé** (ce ne sont pas les messages du défi) de deux messages de 34 caractères chiffrés avec le même flux de clé. Supposons qu'on ait calculé $x = m_1 \oplus m_2$ sans connaître ni m_1 ni m_2. On fait glisser le crib « `␣la` » (4 caractères, avec les espaces) : à chaque position p, on calcule le XOR de $x[p : p+4]$ avec ce crib. Voici quelques résultats :

Position p	Résultat	Verdict
0	`>l/'`	charabia : le crib n'est pas ici
7	`%#q␣`	charabia
19	`>b':`	charabia
23	`n␣e␣`	pourquoi pas…
26	`␣ban`	ressemble à du français : piste sérieuse !

La position 26 donne « `␣ban` », qui pourrait être le début de « banc », « bande », « banque »… On essaie alors d'**allonger** : on devine la suite dans l'autre message (par exemple « `␣banc` »), on

fait le XOR avec x à la même position et on regarde si le premier message reste du français. On alterne ainsi d'un message à l'autre, en agrandissant les fragments petit à petit.

Quelques remarques

- On ne sait pas si le crib appartient à m_1 ou à m_2. Ce n'est pas grave : le XOR est symétrique, on obtient « le morceau de l'autre message ».
- Une fausse piste donne en général des caractères étranges (symboles, caractères non imprimables). Faites filtrer ces résultats par votre programme.
- Les espaces sont précieux : un mot entouré d'espaces (« `que` ») est un crib plus fiable que le mot seul (« `que` »).
- Le début d'une phrase est souvent facile à deviner (« `Le` », « `La` », « `Il` »...) et la fin se termine souvent par un point.

6.5 Une démarche possible

1. Lire c_1 et c_2, puis calculer $x = c_1 \oplus c_2$.
2. Écrire une fonction qui fait glisser un crib sur toutes les positions et n'affiche que les résultats « lisibles » (lettres, espaces, ponctuation simple).
3. Lancer cette fonction avec plusieurs mots français fréquents et noter les positions prometteuses.
4. Écrire une fonction qui teste un crib à une position précise, pour agrandir les fragments un par un (cette partie peut être manuelle : c'est vous qui choisissez les cribs).
5. Quand un message est entièrement retrouvé, en déduire l'autre, puis **vérifier** : retrouvez le flux de clé $k = c_1 \oplus m_1$ et contrôlez que $m_2 \oplus k = c_2$.

7 Ce qu'il faut rendre

Question	Livrables
1 (8 pts)	`image_ecb.bmp`, `image_cbc.bmp`, votre programme et une explication (appuyée sur la définition du mode ECB) de la raison pour laquelle l'une des deux images révèle encore la structure de l'image originale.
2 (7 pts)	Les deux messages retrouvés, votre programme et une courte explication de votre démarche (quels cribs, à quelles positions, comment vous les avez agrandis).

Pour aller plus loin : documentation officielle

- Python, `cryptography` : https://cryptography.io/en/latest/hazmat/primitives/symmetric-encryption/
- Java, `javax.crypto.Cipher` : https://docs.oracle.com/en/java/javase/21/docs/api/java.base/javax/crypto/Cipher.html
- C++, Crypto++ : https://www.cryptopp.com/wiki/Advanced_Encryption_Standard
- C, OpenSSL : https://wiki.openssl.org/index.php/EVP_Symmetric_Encryption_and_Decryption